

AI Travel Harness

A **zero-trust framework that an untrusted AI agent lives inside**. Five agents plan a trip — Flight · Hotel · Car · Experience · Food — but **the agents are the product; the harness is what is being defended and judged**.

An agent proposes an **ID** and nothing more. The harness independently re-grades every claim against data the agent does not control, builds every link itself, persists all state, and raises a structured alarm on every violation.

5 layers 5 untrusted agents 4 harness pillars 27 passing adversarial tests 100% offline & deterministic

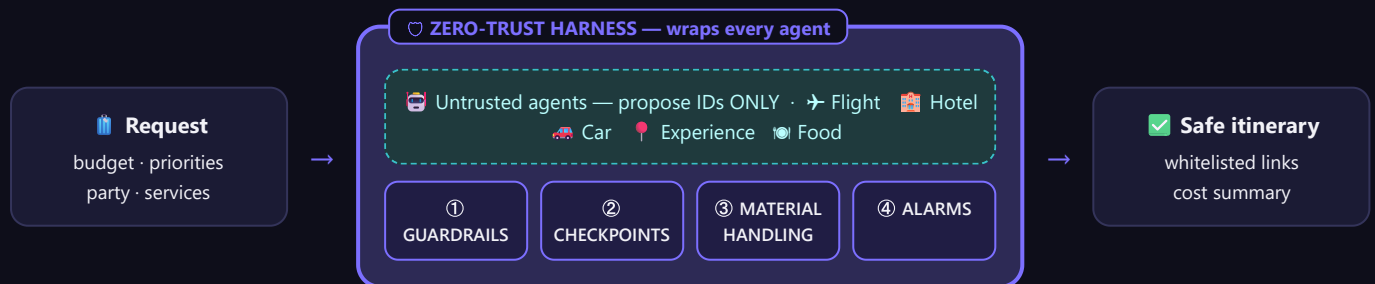
1 The real-world problem

Agentic systems are useful **because they take action** — they book, link, and spend real money. That is also their threat surface. You cannot fix this by asking the model to "be careful." The agent's word can never be the final word.

WHAT THE AGENT DOES WRONG	VECTOR	HARNESS CONTROL
Prompt injection via trip data	malicious string in a text field	Structural sandbox — raw text never reaches an agent; only typed, range-checked fields do
Phishing / link hallucination	agent emits <code>http://evil.tld</code>	Agents emit IDs only ; <code>NoUrlModel</code> rejects URLs; harness mints links from a whitelist
Bad / unsafe booking	\$100 hotel at 2.5★, fake inventory, inflated claims	Shadow Auditor re-grades every claim vs ground-truth inventory
Cross-agent data leakage	Flight agent reads dietary / medical prefs	Material Handling routes only scoped fields; money agents never see prefs
Runaway cost	infinite fallback / retry loop	Economic Governor — hard token + USD ceiling per session
Silent failure	fabricates a result to avoid an empty one	Checkpoints + 3-strike human escalation — never a fabricated booking

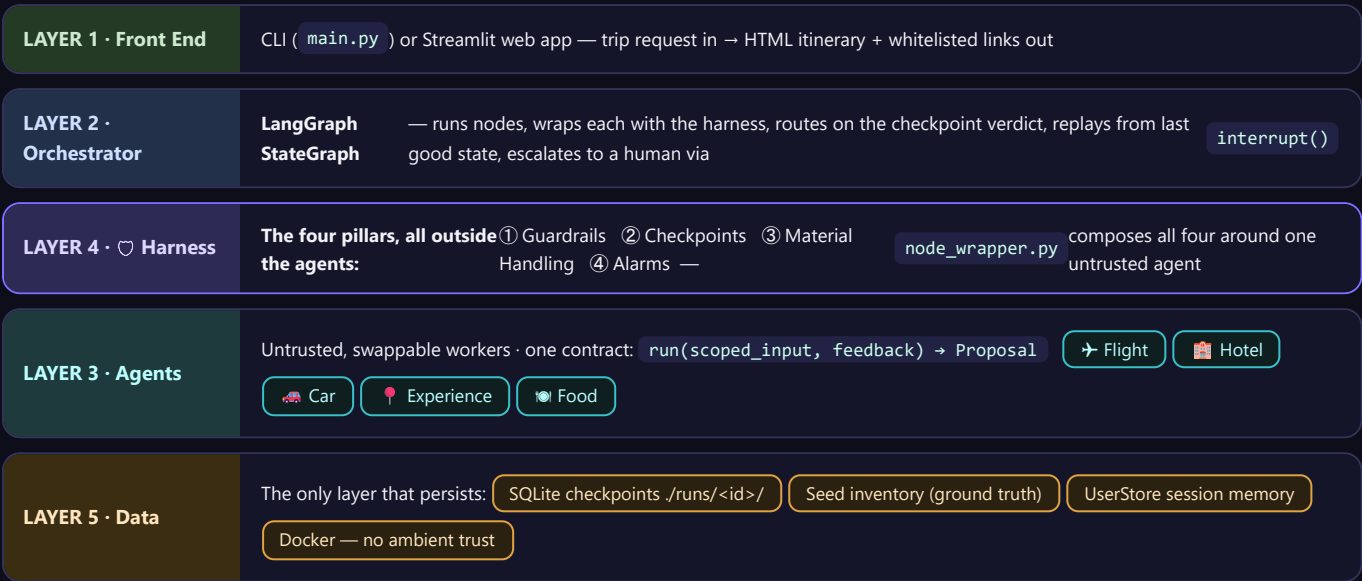
2 The idea — wrap the agent in a harness

All authority — input validation, link minting, state, telemetry — is moved **out of the agent and into a deterministic harness that surrounds it**. The harness **encloses** the agent; it is not a step after it.



3 System architecture — five layers

Nothing crosses a layer boundary unvalidated. Layer 4 (the harness) sits in the gap between an agent's *proposal* and a *booking*.



Technology — and what each piece earns us

CONCERN	BUILT WITH	WHY
Orchestration	LangGraph <code>StateGraph</code>	state machine + checkpointing + replay + <code>interrupt()</code> for free
Trust boundary	Pydantic strict models	the contract is <i>declared in types</i> — an injection string can't become an instruction
Workers	Python (template / Claude swap-in)	one <code>Protocol</code> ; provider-agnostic — Claude <code>claude-opus-4-8</code> earns no extra trust
Telemetry	structured JSON → Splunk (HEC)	live security observability for an AI system
Runtime	Docker (non-root, <code>--network none</code>)	proves zero ambient trust — the seeded demo needs no network egress

4 The four pillars — what the agent gets for free

① Guardrails

- **Input sandbox** — raw request → strict `TripProfile` (Pydantic); only typed, range-checked fields flow on.
- **IDs-not-URLs** — every proposal subclasses `NoUrlModel` ; any URL is rejected.
- **Shadow Auditor** — a 2nd deterministic grader re-checks each claim against ground-truth inventory (rating ≥ floor, price ≤ budget, claim == reality, dietary gate).
- **Economic + Budget Guardrail** — token/\$ ceilings; caps/floors keep the budget split realistic.

② Checkpoints

- Explicit **pass/fail gate after every node** (schema valid + auditor passed), with the tripped criterion broken out for telemetry.
- State persists **only on PASS** → a failed node **replays from the last good checkpoint**; upstream work is never recomputed.
- **3 strikes → human escalation**, never a fabricated booking.

③ Material Handling

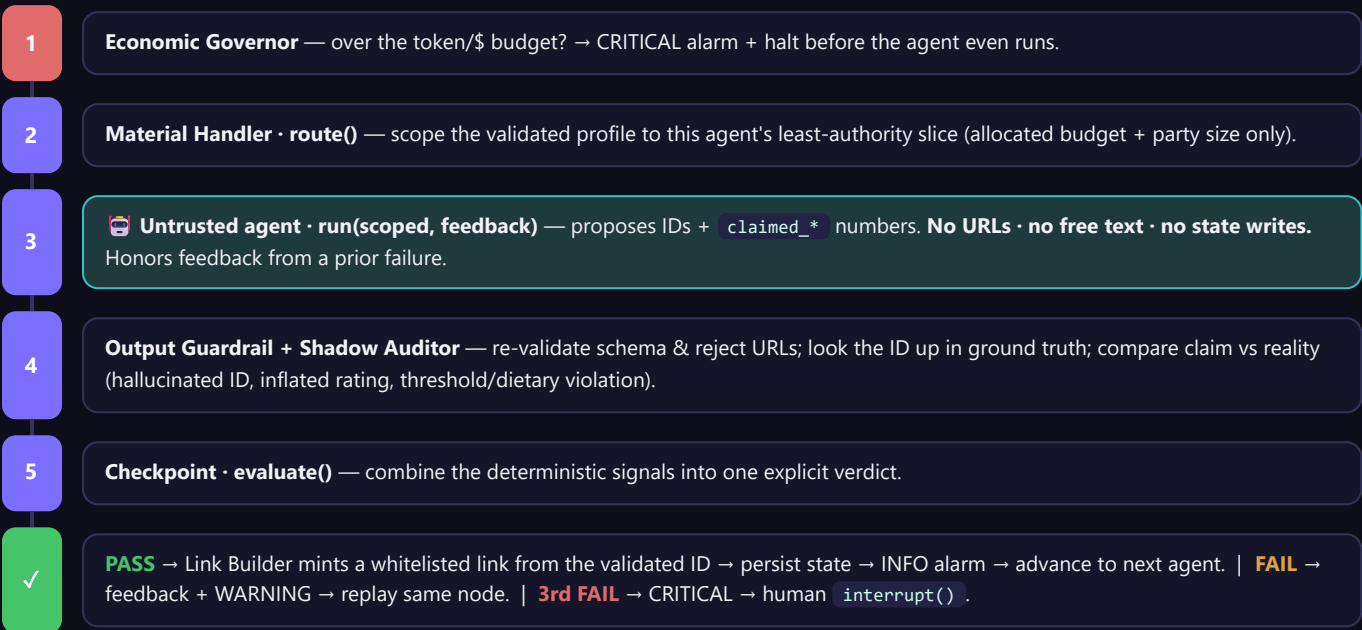
- A declared `SCOPE` table **is** the authority policy — clean interfaces in and out.
- `route()` projects the profile down to one agent's typed slice; `assert_scoped()` is a hard invariant.
- **Least authority by construction** — money agents never receive `diet / cuisine / activities` ; the Flight agent can't leak data it never sees.

④ Alarms

- Every tripped rule emits one structured JSON record: `type · severity · context · recommended_action` .
- Severity = `CRITICAL` / `WARNING` / `INFO`; alarms only **observe** — control flow is decided by the checkpoint logic.
- **Splunk-ready** — set `SPLUNK_HEC_URL/TOKEN` for a live HTTP Event Collector sink (stdlib, no SDK).

5 How the agents work & communicate — the per-agent gauntlet

Agents never talk to each other and never touch shared state. **All communication flows through the harness.** Every agent runs this identical gauntlet, so an agent gains no authority by being "smarter."



The two communication rules that make it zero-trust:
✗ raw user text never reaches an agent · ✗ agents never emit URLs or write state · ✗ money agents never see diet / cuisine / activities.
Inbound = typed scoped slice; **outbound** = opaque IDs + claims, independently re-graded.

6 How the final outcome is produced

STAGE	WHAT THE HARNESS DOES
allocate	Split one total_budget by priority; Budget Guardrail caps/floors it; if preferences > budget → pause pre-booking (accept overage / apply cuts).
agents	Run only the included services (drive own car → no flight/car: \$0, never execute), each through the \$5 gauntlet.
reconcile	Sum the actual validated costs vs budget; a residual overrun is a WARNING, never a hidden charge.
assemble	Render the safe HTML itinerary — harness-built whitelisted links + cost summary. Or → human escalation, never a fabrication.

7 Hackathon requirements → how we delivered

The brief: "Build a harness — a framework an AI agent lives inside, defining what the agent gets for free: guardrails that constrain behavior, checkpoints that evaluate outputs, clean interfaces for passing material in and out, and alarms that fire when something goes wrong."

REQUIREMENT	HOW THE AI TRAVEL HARNESS DELIVERS IT	
A framework the agent lives inside	<code>node_wrapper.make_harness_node()</code> composes all four pillars around every agent; LangGraph runs the wrapped nodes. Agents implement one <code>Protocol</code> and get validation, links, state & telemetry for free.	✓
Guardrails that constrain behavior	Pydantic input sandbox + <code>NoUrlModel</code> output + Shadow Auditor regrade + Economic & Budget guardrails. The agent may only propose an ID; everything else is denied by construction.	✓
Checkpoints that evaluate outputs	An explicit pass/fail gate after every node; state persists only on PASS; failures replay from the last good checkpoint; 3 strikes → human-in-the-loop.	✓
Clean interfaces in & out	Material Handling : a declared <code>SCOPE</code> table routes a typed, least-authority slice in and accepts only IDs out — <code>assert_scoped()</code> proves no out-of-scope field crosses.	✓
Alarms that fire on trouble	A structured-JSON AlarmBus (<code>type · severity · context · action</code>) with pluggable sinks — stderr, JSONL file, and a live Splunk HEC sink.	✓

Beyond the brief: a coordinator **Budget Allocator** with a pre-booking overage interrupt, **optional agents** (run only what the trip needs), a dedicated **Food agent** with a hard dietary safety gate, and a real **Claude swap-in** — proving the harness is provider-agnostic: **swap any agent and not one line of the harness changes**.

5 layers · 5 agents · 4 pillars · 27 passing adversarial tests · 100% offline & deterministic
Python · LangGraph (state · checkpoint · replay · interrupt) · Pydantic · Docker · Splunk — *Travel is just the demo; the harness is the product.*